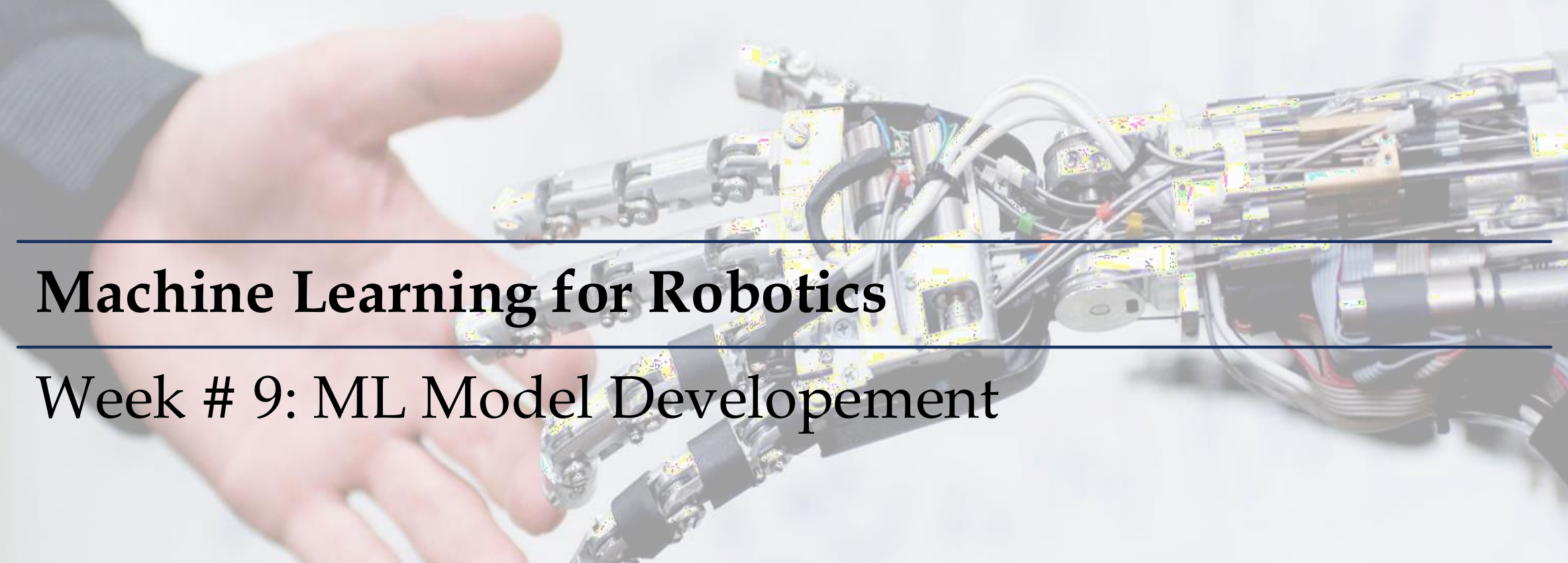


A photograph of a human hand holding a small, intricate robotic arm. The arm is constructed from various metal parts, including gears, joints, and a dense network of colored wires (white, blue, red, yellow). It appears to be a custom-built or experimental robotic device. The background is a plain, light-colored surface.

Machine Learning for Robotics

Week # 9: ML Model Developement

Dr. Maryam Iqbal

ABOUT INSTRUCTOR

- PhD EE (Controls, Automation & Rehabilitation Robotics)
- Research areas: Robotics & Automation, Artificial Intelligence, Machine Learning, Biomechanics, AI in Healthcare
- In charge : BSRIS Program at Bahria University
- Research Publications, Funded Projects and Patents
- Member of AI Task Force (Ministry of Planning, Development & Special Initiatives of Pakistan)
- Senior Member IEEE (Region 10) | General Secretary IEEE RAS (Region 3) | Member IEEE CSS
- Email: maryamiqbal.buic@bahria.edu.pk
- LinkedIn: [linkedin.com/in/maryam-iqbal-3b942a66](https://www.linkedin.com/in/maryam-iqbal-3b942a66)
- Google Scholar: <https://scholar.google.com/citations?user=-4togNoAAAAJ&hl=en>

LECTURE OBJECTIVES

Explore

Explore Steps for ML Model development

Learn

Learn ML Model Evaluation Parameters

Discover

Discover how this knowledge can be utilized in Robotics

ML MODEL LIFE CYCLE

Steps for a ML model development

1. Data Collection
2. Preprocessing Data
3. Selection of ML algorithm
4. Model Training
5. Model Evaluation
6. Model Testing
7. Model Deployment

DATA COLLECTION

Dataset

- Dataset is essentially the backbone for all operations, techniques or models
- Datasets are used to train and test AI models
- They provide the raw material for computers to learn patterns and make predictions
- Datasets involve a large amount of data points grouped into one table
- In a dataset, the rows represent the number of data points and the columns represent the features of the Dataset
- Datasets may require cleaning and preprocessing to ensure data quality.
- Datasets can be stored in multiple formats. The most common ones are .txt, CSV, Excel, JSON, and zip files for large datasets.

DATA COLLECTION

Types of Datasets

Training Dataset: It is used to train the machine learning model

Validation set: A subset of the dataset that is used evaluate performance during training of the model.

Test Set: It is used to evaluate final model performance on unseen data

Numerical Dataset: It includes temperature, marks etc.

Categorical Dataset: It includes categories such as colour, gender, occupation

Image Dataset: It includes a dataset consisting of images. This is mostly used to differentiate the types of diseases, heart conditions

Ordered Dataset: These datasets contain data that are ordered in ranks, for example, customer reviews, movie ratings

Partitioned Dataset: These datasets have data points segregated into different partitions.

DATA COLLECTION

Dataset collection/construction

- **Online Repositories:** Websites like Kaggle, UCI (University of California, Irvine), Machine Learning Repository offer a vast collection of public datasets for various domains.
- **Government Agencies:** Government agencies often release public datasets on topics like economics, healthcare, and environment.
- **Research Institutions:** Universities and research organizations may share their datasets for public use.
- **Novel dataset construction:** Novel dataset can be constructed like taking MRI images of brain tumor, Skin cancer, or any other kind of cancer from hospitals

PREPROCESSING DATA

Data Cleaning:

- **Handling Missing Values:** Replace missing values with appropriate values (e.g., mean, median, mode) or remove rows or columns with too many missing values.
- **Correcting Errors:** Identify and correct any inconsistencies or errors in the data, such as typos or incorrect data formats.

Data Transformation:

- **Normalization:** Scale numerical features to a specific range (e.g., 0-1 or -1 to 1) to ensure they have a similar impact on the model.
- **Feature Engineering/Encoding:** Convert sequential data into numerical representations (e.g., one-hot encoding)

Data Reduction:

- **Feature Selection:** Select the most relevant features to improve model efficiency.
- **Dimensionality Reduction:** Use techniques like Principal Component Analysis (PCA) to reduce the number of features while preserving important information.

SELECTION OF ML ALGORITHM

Selection of ML algorithm

- Selecting the right ML algorithm plays a pivotal role in building of successful model
- Understanding the nature of the problem like classification, regression, or clustering
- Understanding a variety of machine learning algorithms suitable for your problem type
- The best approach is often to experiment with multiple models, evaluate their metrics and generalization to unseen/testing data

Training Model

- Input the preprocessed data into the selected ML algorithm
- ML algorithm learns patterns, features, and associations from the data
- Optimization techniques like grid search can be used to tune the hyperparameters.

MODEL EVALUATION

Model Evaluation:

Various metrics used to evaluate model performance

For classification tasks, the following metrics are used

	Predicted Dog	Predicted Not Dog
Actual Dog	True Positive (TP)	False Negative (FN)
Actual Not Dog	False Positive (FP)	True Negative (TN)

Confusion Matrix:

It is a matrix that summarizes the performance of a classification model

It calculates true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) instances

Confusion Matrix For binary classification:

A 2X2 Confusion matrix is shown below for the image recognition having a Dog image or Not Dog image

True Positive (TP): It is the total counts having both predicted and actual values are Dog.

True Negative (TN): It is the total counts having both predicted and actual values are Not Dog.

False Positive (FP): It is the total counts having prediction is Dog while actually Not Dog.

False Negative (FN): It is the total counts having prediction is Not Dog while actually, it is Dog.

EVALUATION METRICS

- Actual Dog Counts = 6
- Actual Not Dog Counts = 4
- True Positive Counts = 5
- False Positive Counts = 1
- True Negative Counts = 3
- False Negative Counts = 1

	Predicted Dog	Predicted Not Dog
Actual Dog	TP = 5	FN = 1
Actual Not Dog	FP = 1	TN = 3

Evaluation metrics

1. Accuracy:

- It is used to measure the performance of the model. It is the ratio of Total correct instances to the total instances.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Accuracy = (5+3)/(5+3+1+1) = 8/10 = 0.8

■ 2. Precision

- Precision is a measure of how accurate a model's positive predictions are.
- It is defined as the ratio of true positive predictions to the total number of positive predictions made by the model

$$\text{Precision} = \frac{TP}{TP + FP}$$

- For the above case:
- Precision = $5 / (5 + 1) = 5 / 6 = 0.8333$

■ 3. Recall

- It is the ratio of the number of true positive (TP) instances to the sum of true positive and false negative (FN) instances.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- For the above case:
- Recall = $5 / (5 + 1) = 5 / 6 = 0.8333$

4. F1-Score

- It is used to evaluate the overall performance of a classification model.
- Harmonic mean of **Precision** and **Recall**
- Balances both metrics into a single score

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- For the above case:
- F1-Score: = $(2 \cdot 0.8333 \cdot 0.8333) / (0.8333 + 0.8333) = 0.8333$

5. Specificity

- It measures the ability of a model to correctly identify negative instances.
- It is also known as the True Negative Rate.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

- For example,
- Specificity = $3 / (1 + 3) = 3 / 4 = 0.75$

CONFUSION MATRIX FOR MULTICLASS PROBLEM

Confusion Matrix for 3-classes (Dog, Cat, Camel) problem

- Dog=58, Cat=70, Camel=79
- Compute confusion matrix for each class, also called one vs all confusion matrix

Actual \ Predicted	Dog	Cat	Camel
Dog	50	5	3
Cat	4	60	6
Camel	2	7	70

Actual \ Predicted	Dog	Not Dog (Cat + Camel)
Dog	TP = 50	FN = 8
Not Dog	FP = 6	TN = 143



Actual \ Predicted	Cat	Not Cat (Dog + Camel)
Cat	TP = 60	FN = 10
Not Cat	FP = 12	TN = 125

Actual \ Predicted	Camel	Not Camel (Dog + Cat)
Camel	TP = 70	FN = 9
Not Camel	FP = 9	TN = 119

MODEL EVALUATION

Fold	Training Set	Test Set
1st	{3, 4, 5, 6, 7, 8, 9, 10}	{1, 2}
2nd	{1, 2, 5, 6, 7, 8, 9, 10}	{3, 4}
3rd	{1, 2, 3, 4, 7, 8, 9, 10}	{5, 6}
4th	{1, 2, 3, 4, 5, 6, 9, 10}	{7, 8}
5th	{1, 2, 3, 4, 5, 6, 7, 8}	{9, 10}

Model cross validation methods

(1) K-fold (2) Leave-One-Out, (3) Hold-out validation

1) K-fold Cross-Validation:

- Divide the dataset into k equal-sized folds.
- Train the model on k-1 folds and evaluate on the remaining fold.
- Repeat k times, using each fold for validation once.
- Average the performance metrics across all iterations
- **Example (k=5):**
- Consider **10 data points**:
- {1,2,3,4,5,6,7,8,9,10} We divide them into 5 folds:

2) Leave-One-Out Cross-Validation

A special case of k-fold cross-validation where k equals the number of samples in the dataset.

Leave one sample out for validation in each iteration.

Train the model on the remaining samples and evaluate on the left-out sample.

Repeat for all samples

Example:

Dataset: {1,2,3,4,5}



3. Hold-Out Validation

This is the simplest method where the dataset is divided into two sets:

Training set (used to train the model)

Test set (used to evaluate the model)

Numerical Example:

Imagine we have **10 data points**:

80% Training Set: {1,2,3,4,5,6,7,8}

20% Test Set: {9,10}



Model Testing



After model construction, it is evaluated by unseen/testing dataset.



It determine the generalization power of the model



It shows the overall model performance



Model Deployment



During model deployment, it's essential to ensure that the system can handle high user loads



Operate smoothly without crash\hang, and be easily updated.

ACTIVITY

- **Supervised Learning – Fraud Detection**
- **Goal:** Identify fraudulent transactions.
- **Method:**
- Use classification algorithms (e.g., Logistic Regression, Random Forest, SVM).
- Train on labelled data → fraudulent (1) vs. legitimate (0).
- Model learns transaction patterns to predict new cases.
- **Example Features:** Transaction amount, time, location, frequency.

ACTIVITY

- **UnSupervised Learning – Customer Segmentation**
- **Goal:** Group customers based on behaviour.
- **Method:**
- Use clustering algorithms (e.g., K-Means, DBSCAN, Hierarchical).
- No labels provided → algorithm finds hidden structures in data.
- Helps businesses in targeted marketing & personalized offers.
- **Example Features:** Purchase history, age, spending habits.

EXPLANATION OF CODE

- This code creates a fake fraud detection dataset with 5,000 transactions, 6 features, and a fraud label (0 or 1).
- It simulates real-world imbalanced data where fraud is much less frequent than normal transactions.

Create a sample fraud dataset

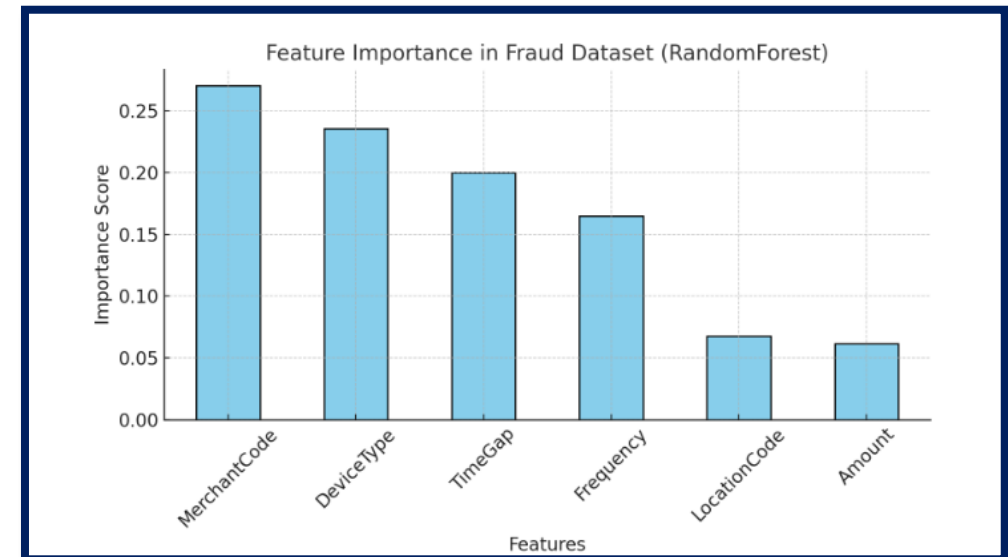
```
from sklearn.datasets import make_classification
```

- **Explanation**
- The comment # means “this is just an explanation.”
- “We are importing make_classification from sklearn.datasets.”
- make_classification is a helper function to create fake (synthetic) datasets for testing ML models.

```
X, y = make_classification(  
    n_samples=5000,      # total number of rows (transactions)  
    n_features=6,        # number of input features (columns of data)  
    n_informative=4,     # how many features actually help in classification  
    n_redundant=0,       # how many features are just random noise  
    n_classes=2,         # we want 2 categories: Fraud (1) and Not Fraud (0)  
    weights=[0.9, 0.1],  # imbalance: 90% normal, 10% fraud (realistic scenario)  
    random_state=42      # fixed seed, so results are the same every time  
)
```

X → contains the features (like transaction amount, time, location).

y → contains the labels (0 = not fraud, 1 = fraud).



```
df_fraud = pd.DataFrame(  
    X,  
    columns=['Amount','TimeGap','LocationCode','DeviceType','Frequency','MerchantCode']  
)
```

- Converts X (the features) into a Pandas DataFrame (table format, like Excel).
- Gives nice column names instead of just numbers.
- Example features: Amount → transaction amount.
- TimeGap → time difference from last transaction.
- LocationCode → encoded location ID.
- DeviceType → type of device used.
- Frequency → how often user transacts.
- MerchantCode → merchant identifier.

```
df_fraud['Fraudulent'] = y
```

- Adds the target label (y) as a new column.
- Now each row has both: Transaction details (X)Fraud/Not Fraud label (y).

```
print(df_fraud.head())
```

- Prints the first 5 rows of the dataset to check what it looks like.

Step 1: Split the dataset

```
X = df_fraud.drop('Fraudulent', axis=1) # all columns except Fraudulent
```

```
y = df_fraud['Fraudulent']           # only the target column
```

`df_fraud` = our full dataset (features + labels).

- `X` = all features (Amount, TimeGap, DeviceType, etc.), i.e., input data.
- `y` = the Fraudulent column (0 = not fraud, 1 = fraud), i.e., target labels.
- `drop('Fraudulent', axis=1)` removes that column from `X`.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- Splits the dataset into training (80%) and testing (20%).
- `X_train, y_train` → used to train the model.
- `X_test, y_test` → used to evaluate the model.
- `random_state=42` → ensures reproducibility (same split every time).

X will look like:

Amount	Time_of_Day	Location
200	Morning	Online
5000	Night	Abroad
120	Afternoon	Local

y will look like:

Fraudulent

0

1

0

ML model learns patterns in features (large amount + night + abroad → high chance of fraud) instead of directly reading the Fraudulent column.

Step 2: Train the Model

```
model = RandomForestClassifier(n_estimators=100, random_state=42)  
model.fit(X_train, y_train)
```

- RandomForestClassifier = an ML algorithm (ensemble of many decision trees).
- n_estimators=100 → model builds 100 decision trees and averages their results.
- fit(X_train, y_train) → trains the model using the training data.

Step 3: Make Predictions

```
y_pred = model.predict(X_test)
```

- The trained model predicts, fraud (1) or not fraud (0) for the unseen test data (X_test).
- y_pred = model's predictions.

Step 4: Evaluate Performance

```
print(classification_report(y_test, y_pred))
```

- Prints precision, recall, f1-score, and accuracy. These metrics show how well the model detected frauds vs normal transactions.

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Blues")  
plt.title("Fraud Detection Confusion Matrix")  
plt.show()
```

- `confusion_matrix` = table showing:
- True Positives (TP): fraud correctly detected.
- True Negatives (TN): normal correctly detected.
- False Positives (FP): predicted fraud, but it was normal.
- False Negatives (FN): missed fraud.
- `sns.heatmap(...)` → makes it look like a nice colored grid. The confusion matrix is very important in fraud detection, because missing fraud (FN) is worse than wrongly flagging a normal case.

CONFUSION MATRIX

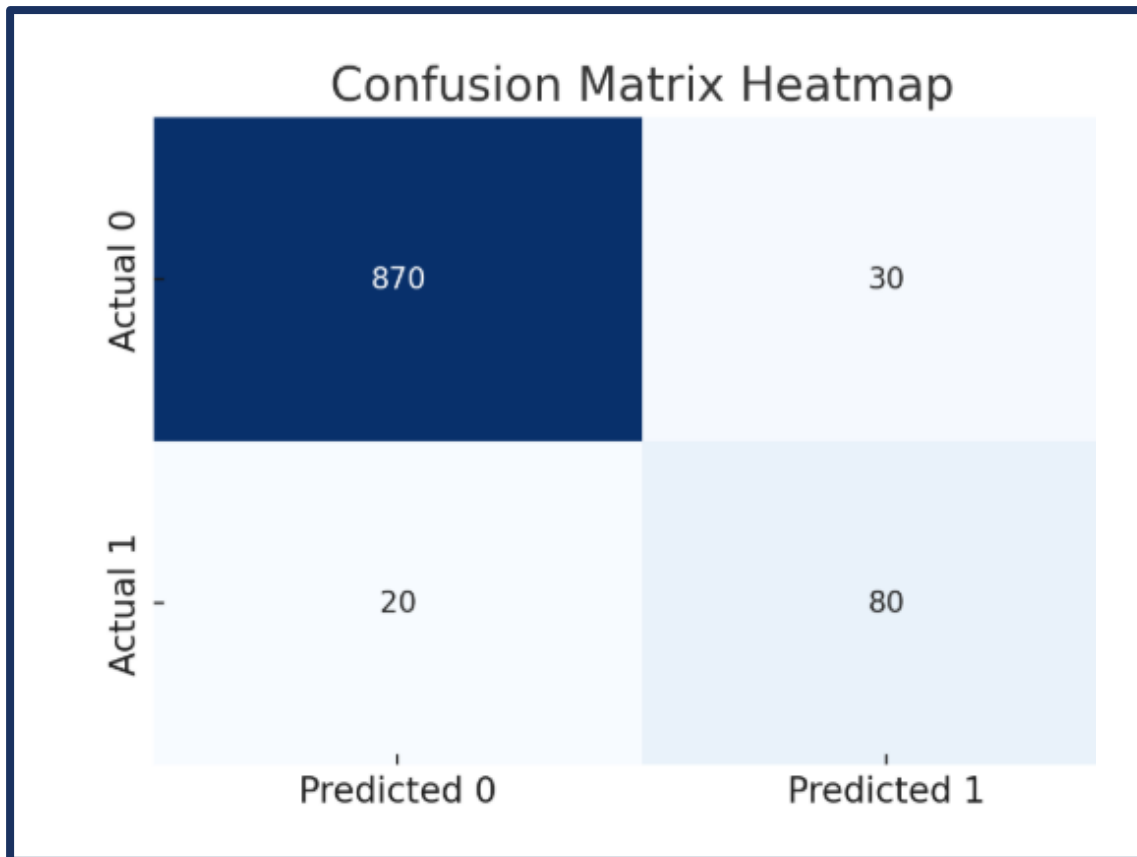
Confusion Matrix Example

	Predicted Not Fraud (0)	Predicted Fraud (1)
Actual Not Fraud (0)	870 (True Negative, TN)	30 (False Positive, FP)
Actual Fraud (1)	20 (False Negative, FN)	80 (True Positive, TP)

Interpretation:

- TN (870): 870 normal transactions correctly predicted as normal.
- FP (30): 30 normal transactions wrongly flagged as fraud.
- FN (20): 20 fraud cases missed (predicted as normal 😬).
- TP (80): 80 fraud cases correctly identified ✅.

CONFUSION MATRIX



- Darker blue boxes = higher numbers.
- Diagonal values (870 & 80) = correct predictions.
- Off-diagonal values (30 & 20) = errors.

1. *Accuracy*: How many **predictions** are correct? overall?

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{\text{Total}} \\ &= (80 + 870) \div 1000 = 95\% \end{aligned}$$

- High accuracy looks good — but in fraud detection, it can be misleading (because fraud is rare).

2. *Precision (of fraud predictions)*: When the model says “fraud,” how often is it correct?

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP} \\ &= 80 \div (80 + 30) = 72.7\% \end{aligned}$$

- **Means ~73% of flagged transactions were truly fraud.**
- If precision is low → too many false alarms.

3. *Recall (Sensitivity):*

Out of all actual frauds, how many did we catch?

$$Recall = \frac{TP}{TP + FN}$$

$$= 80 \div (80 + 20) = 80\%$$

- Means we caught 80% of fraud cases.
- If recall is low → too many frauds slip through (dangerous).

4. *F1-Score: Balance between Precision and Recall.*

$$F - 1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$= 2 \times (0.727 \times 0.80) \div (0.727 + 0.80) \approx 76\%$$

- A good single metric when fraud data is imbalanced.
- Precision = “How trustworthy are fraud alerts?” Recall = “How many frauds are we actually catching?” F1-Score = “Overall balance between the two.”

QUESTIONS

